

Депонируемые материалы

Программа для ЭВМ «Tokamak Analysis»

ДЕПОНИРУЕМЫЕ МАТЕРИАЛЫ

Программа для ЭВМ: Tokamak Analysis — ИИ-система для анализа
экспериментальных данных токамаков

Автор(ы): [ФИО авторов]

Правообладатель: НИЯУ МИФИ

Дата создания: 2026

Дата формирования листинга: 05.05.2026

Языки программирования: Python 3.11, TypeScript 5.6

Объем: ~14 000 строк исходного кода

ФАЙЛ: backend/app/main.py

Описание: Точка входа серверного приложения (FastAPI), middleware, маршруты

```
from __future__ import annotations

import time
from contextlib import asynccontextmanager

import structlog
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse, RedirectResponse
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded
from slowapi.util import get_remote_address
from starlette.middleware.base import BaseHTTPMiddleware, RequestResponseEndpoint
from starlette.responses import Response

from app.api import admin, auth, experiments, legal, models, predictions
from app.config import settings
from app.core.headers import SecurityHeadersMiddleware
from app.core.logging_config import setup_logging
from app.core.metrics import instrumentator
from app.services.ml_client import MLServiceClient

logger = structlog.get_logger()

class RequestLoggingMiddleware(BaseHTTPMiddleware):
    """Log method, path, status code and duration for every request."""

    async def dispatch(self, request: Request, call_next: RequestResponseEndpoint)
-> Response:
        start = time.perf_counter()
        response = await call_next(request)
        duration_ms = round((time.perf_counter() - start) * 1000, 2)
```

```

        await logger.ainfo(
            "request",
            method=request.method,
            path=request.url.path,
            status=response.status_code,
            duration_ms=duration_ms,
        )
    return response

```

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    setup_logging()
    from app.database import engine
    yield
    # Shutdown
    await engine.dispose()

```

```

limiter = Limiter(key_func=get_remote_address,
default_limits=[f"{settings.rate_limit_per_minute}/minute"])

```

```

app = FastAPI(
    title="Tokamak Analysis API",
    description="ИИ-система для анализа экспериментальных данных токамаков",
    version="1.0.0",
    lifespan=lifespan,
)

```

```

app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

```

```

@app.exception_handler(RequestValidationError)
async def validation_error_handler(request: Request, exc: RequestValidationError)
-> JSONResponse:
    return JSONResponse(
        status_code=422,
        content={"detail": exc.errors()},
    )

```

```

# Middleware (order matters: outermost middleware is added last)
app.add_middleware(RequestLoggingMiddleware)
app.add_middleware(SecurityHeadersMiddleware)

```

```

# CORS
origins = [o.strip() for o in settings.backend_cors_origins.split(",")]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

```

# Routers

```

```

app.include_router(auth.router)
app.include_router(experiments.router)
app.include_router(predictions.router)
app.include_router(models.router)
app.include_router(admin.router)
app.include_router(legal.router)

# Prometheus metrics
instrumentator.instrument(app).expose(app)

@app.get("/api/v1/health")
async def health():
    ml_client = MLServiceClient()
    ml_status = await ml_client.health()
    return {
        "status": "healthy",
        "version": "1.0.0",
        "ml_service": "connected" if ml_status else "unavailable",
    }

@app.get("/api/docs")
async def api_docs_redirect():
    return RedirectResponse(url="/docs")

@app.get("/health")
async def health_simple():
    return {"status": "ok"}

```

```

=====
ФАЙЛ: backend/app/api/auth.py
Описание: API аутентификации (регистрация, JWT, 152-ФЗ)
=====

```

```

from __future__ import annotations

from datetime import datetime, timezone

from fastapi import APIRouter, Depends, HTTPException, Request, status
from sqlalchemy import delete, select
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.audit import log_action
from app.core.security import (
    create_access_token,
    create_refresh_token,
    decode_token,
    get_current_user,
    hash_password,
    verify_password,
)
from app.database import get_db
from app.models.audit_log import AuditLog
from app.models.experiment import Experiment, TimeSeriesData
from app.models.ml_model import ModelRun
from app.models.prediction import Prediction

```

```

from app.models.user import User
from app.schemas.auth import (
    TokenRefresh,
    TokenResponse,
    UserCreate,
    UserLogin,
    UserResponse,
)

router = APIRouter(prefix="/api/v1/auth", tags=["auth"])

@router.post("/register", response_model=UserResponse,
status_code=status.HTTP_201_CREATED)
async def register(
    data: UserCreate,
    request: Request,
    db: AsyncSession = Depends(get_db),
):
    if not data.consent_given:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Необходимо согласие на обработку персональных данных (152-ФЗ)",
        )

    existing = await db.execute(select(User).where(User.email == data.email))
    if existing.scalar_one_or_none():
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail="Email
already registered")

    user = User(
        email=data.email,
        password_hash=hash_password(data.password),
        full_name=data.full_name,
        role=data.role,
        consent_given_at=datetime.now(timezone.utc),
    )
    db.add(user)
    await db.flush()

    await log_action(db, "register", "user", user_id=user.id, request=request)
    return user

@router.post("/login", response_model=TokenResponse)
async def login(
    data: UserLogin,
    request: Request,
    db: AsyncSession = Depends(get_db),
):
    result = await db.execute(select(User).where(User.email == data.email))
    user = result.scalar_one_or_none()

    if not user or not verify_password(data.password, user.password_hash):
        await log_action(
            db, "login_failed", "auth",
            details={"email": data.email},
            request=request,

```

```

        )
        await db.commit()
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid credentials")

    if not user.is_active:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Account
disabled")

    await log_action(db, "login", "auth", user_id=user.id, request=request)

    return TokenResponse(
        access_token=create_access_token(user.id, user.role.value),
        refresh_token=create_refresh_token(user.id),
    )

@router.post("/refresh", response_model=TokenResponse)
async def refresh_token(
    data: TokenRefresh,
    db: AsyncSession = Depends(get_db),
):
    payload = decode_token(data.refresh_token)
    if payload.get("type") != "refresh":
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid token type")

    user_id = int(payload["sub"])
    result = await db.execute(select(User).where(User.id == user_id,
User.is_active.is_(True)))
    user = result.scalar_one_or_none()
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="User
not found")

    return TokenResponse(
        access_token=create_access_token(user.id, user.role.value),
        refresh_token=create_refresh_token(user.id),
    )

@router.get("/me", response_model=UserResponse)
async def get_me(current_user: User = Depends(get_current_user)):
    return current_user

@router.delete("/account", status_code=status.HTTP_200_OK)
async def delete_account(
    request: Request,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    """Delete current user account and all associated data (152-Ø3, ст. 21)."""
    user_id = current_user.id

    # Delete predictions linked to user's experiments
    user_experiment_ids = select(Experiment.id).where(Experiment.user_id ==
user_id)

```

```

    await db.execute(
        delete(Prediction).where(Prediction.experiment_id.in_(user_experiment_ids))
    )

    # Delete timeseries data linked to user's experiments
    await db.execute(
delete(TimeSeriesData).where(TimeSeriesData.experiment_id.in_(user_experiment_ids))
    )

    # Delete user's experiments
    await db.execute(delete(Experiment).where(Experiment.user_id == user_id))

    # Delete user's model runs
    await db.execute(delete(ModelRun).where(ModelRun.user_id == user_id))

    # Delete user's audit logs
    await db.execute(delete(AuditLog).where(AuditLog.user_id == user_id))

    # Soft-delete the user account
    current_user.is_active = False
    current_user.deleted_at = datetime.now(timezone.utc)

    await log_action(
        db, "account_deleted", "user",
        user_id=user_id,
        details={"email": current_user.email},
        request=request,
    )
    await db.commit()

    return {"detail": "Аккаунт и все связанные данные удалены"}

@router.get("/export-data")
async def export_data(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    """Export all user personal data (152-ФЗ, ст. 14 / right to data
portability)."""
    # Profile
    profile = {
        "id": current_user.id,
        "email": current_user.email,
        "full_name": current_user.full_name,
        "role": current_user.role.value,
        "is_active": current_user.is_active,
        "created_at": current_user.created_at.isoformat() if
current_user.created_at else None,
        "consent_given_at": current_user.consent_given_at.isoformat() if
current_user.consent_given_at else None,
    }

    # Experiments
    result = await db.execute(
        select(Experiment).where(Experiment.user_id == current_user.id)
    )

```

```

experiments = result.scalars().all()
experiments_data = [
    {
        "id": exp.id,
        "shot_id": exp.shot_id,
        "source": exp.source.value,
        "status": exp.status.value,
        "metadata": exp.metadata_json,
        "loaded_at": exp.loaded_at.isoformat() if exp.loaded_at else None,
    }
    for exp in experiments
]

# Predictions linked to user's experiments
experiment_ids = [exp.id for exp in experiments]
predictions_data = []
if experiment_ids:
    result = await db.execute(
        select(Prediction).where(Prediction.experiment_id.in_(experiment_ids))
    )
    predictions = result.scalars().all()
    predictions_data = [
        {
            "id": pred.id,
            "experiment_id": pred.experiment_id,
            "model_id": pred.model_id,
            "result": pred.result_json,
            "probability": pred.probability,
            "created_at": pred.created_at.isoformat() if pred.created_at else
None,
        }
        for pred in predictions
    ]

# Model runs
result = await db.execute(
    select(ModelRun).where(ModelRun.user_id == current_user.id)
)
model_runs = result.scalars().all()
model_runs_data = [
    {
        "id": run.id,
        "model_id": run.model_id,
        "status": run.status.value,
        "hyperparams": run.hyperparams_json,
        "metrics": run.metrics_json,
        "progress": run.progress,
        "started_at": run.started_at.isoformat() if run.started_at else None,
        "finished_at": run.finished_at.isoformat() if run.finished_at else
None,
        "created_at": run.created_at.isoformat() if run.created_at else None,
    }
    for run in model_runs
]

return {
    "profile": profile,
    "experiments": experiments_data,

```

```
        "predictions": predictions_data,
        "model_runs": model_runs_data,
    }
```

```
=====
ФАЙЛ: backend/app/core/security.py
```

```
Описание: Аутентификация JWT, хеширование паролей (bcrypt), RBAC
=====
```

```
from __future__ import annotations
```

```
from datetime import datetime, timedelta, timezone
```

```
from fastapi import Depends, HTTPException, Request, status
```

```
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
```

```
from jose import JWTError, jwt
```

```
from passlib.context import CryptContext
```

```
from sqlalchemy import select
```

```
from sqlalchemy.ext.asyncio import AsyncSession
```

```
from app.config import settings
```

```
from app.database import get_db
```

```
from app.models.user import User, UserRole
```

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto",
```

```
bcrypt__rounds=12)
```

```
security_scheme = HTTPBearer()
```

```
def hash_password(password: str) -> str:
```

```
    return pwd_context.hash(password)
```

```
def verify_password(plain_password: str, hashed_password: str) -> bool:
```

```
    return pwd_context.verify(plain_password, hashed_password)
```

```
def create_access_token(user_id: int, role: str) -> str:
```

```
    expire = datetime.now(timezone.utc) +
```

```
    timedelta(minutes=settings.jwt_access_token_expire_minutes)
```

```
    payload = {"sub": str(user_id), "role": role, "type": "access", "exp": expire}
```

```
    return jwt.encode(payload, settings.jwt_secret_key,
```

```
    algorithm=settings.jwt_algorithm)
```

```
def create_refresh_token(user_id: int) -> str:
```

```
    expire = datetime.now(timezone.utc) +
```

```
    timedelta(days=settings.jwt_refresh_token_expire_days)
```

```
    payload = {"sub": str(user_id), "type": "refresh", "exp": expire}
```

```
    return jwt.encode(payload, settings.jwt_secret_key,
```

```
    algorithm=settings.jwt_algorithm)
```

```
def decode_token(token: str) -> dict:
```

```
    try:
```

```
        return jwt.decode(token, settings.jwt_secret_key,
```

```
        algorithms=[settings.jwt_algorithm])
```



```

        except JWTError as e:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                                detail="Invalid token") from e

    async def get_current_user(
        credentials: HTTPAuthorizationCredentials = Depends(get_security_scheme),
        db: AsyncSession = Depends(get_db),
    ) -> User:
        payload = decode_token(credentials.credentials)
        if payload.get("type") != "access":
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
                                detail="Invalid token type")

        user_id = int(payload["sub"])
        result = await db.execute(select(User).where(User.id == user_id,
            User.is_active.is_(True)))
        user = result.scalar_one_or_none()
        if user is None:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="User
            not found")
        return user

def require_role(*roles: UserRole):
    def checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in roles:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
                                detail="Insufficient permissions")
        return current_user
    return checker

```

=====

ФАЙЛ: ml_service/service/server.py

Описание: ML-сервис (FastAPI) – классификация, предсказание срывов, обучение

=====

```

from __future__ import annotations

import os
import time

import numpy as np
import structlog
from fastapi import FastAPI, HTTPException
from fastapi.responses import Response
from pydantic import BaseModel

from service.metrics import CONTENT_TYPE_LATEST, ML_INFERENCE_DURATION,
ML_TRAINING_DURATION, generate_latest

from service.data.dataset import get_sample_dataset
from service.data.fair_mast_loader import load_disruption_labels, load_shots,
make_disruption_labels
from service.data.metrics import compute_metrics, format_confusion_matrix
from service.data.preprocessing import prepare_batch, prepare_features
from service.training.trainer import create_model, load_model

```

```

logger = structlog.get_logger()

app = FastAPI(title="Tokamak ML Service", version="1.0.0")

# Cache for loaded models
_model_cache: dict[str, object] = {}

class PredictRequest(BaseModel):
    data: dict
    model_path: str
    model_type: str
    threshold: float = 0.7

class TrainRequest(BaseModel):
    model_type: str
    task: str
    hyperparameters: dict = {}
    shot_ids: list[int] = []
    run_id: int | None = None

@app.get("/health")
async def health():
    return {"status": "healthy", "service": "ml"}

@app.get("/metrics")
async def metrics():
    return Response(content=generate_latest(), media_type=CONTENT_TYPE_LATEST)

@app.post("/api/v1/predict/classify")
async def classify(request: PredictRequest):
    try:
        # Prepare features
        features = prepare_features(request.data.get("signals", {}))

        # Load or get cached model
        model = _get_model(request.model_type, request.model_path)

        start = time.perf_counter()
        prediction = model.predict(features)
        proba = model.predict_proba(features)
        inference_ms = (time.perf_counter() - start) * 1000

        label = "stable" if prediction[0] == 0 else "unstable"
        confidence = float(proba[0].max())

        logger.info("classification_done", label=label, confidence=confidence,
inference_ms=inference_ms)

    return {
        "label": label,
        "confidence": confidence,
        "inference_time_ms": inference_ms,
    }

```

```

    }
except Exception as e:
    logger.error("classification_failed", error=str(e))
    raise HTTPException(status_code=500, detail=str(e))

@app.post("/api/v1/predict/disruption")
async def predict_disruption(request: PredictRequest):
    try:
        features = prepare_features(request.data.get("signals", {}))
        model = _get_model(request.model_type, request.model_path)

        start = time.perf_counter()
        proba = model.predict_proba(features)
        inference_ms = (time.perf_counter() - start) * 1000

        probabilities = proba[:, 1].tolist() if proba.ndim > 1 else proba.tolist()

        # Generate timestamps (normalized)
        n_steps = len(probabilities)
        timestamps = list(np.linspace(0, 1, n_steps))

        # Check if warning should be issued
        max_prob = max(probabilities)
        warning_issued = max_prob > request.threshold
        warning_time_ms = None
        if warning_issued:
            warning_idx = next(i for i, p in enumerate(probabilities) if p >
request.threshold)
            warning_time_ms = timestamps[warning_idx] * 1000

        return {
            "timestamps": timestamps,
            "probabilities": probabilities,
            "warning_issued": warning_issued,
            "warning_time_ms": warning_time_ms,
            "max_probability": max_prob,
            "inference_time_ms": inference_ms,
        }
    except Exception as e:
        logger.error("disruption_prediction_failed", error=str(e))
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/api/v1/train")
async def train(request: TrainRequest):
    try:
        logger.info("training_started", model_type=request.model_type,
task=request.task)

        model = create_model(request.model_type, request.hyperparameters or None)

        seq_len = request.hyperparameters.get("sequence_length", 200)
        n_features = request.hyperparameters.get("input_size", 39)

        if request.shot_ids:
            # Load real data from FAIR-MAST
            shots_data, loaded_ids = load_shots(request.shot_ids)

```

```

        if not shots_data:
            raise HTTPException(
                status_code=404,
                detail="No valid shots could be loaded from the provided
shot_ids",
            )
        X = prepare_batch(
            [{"signals": s} for s in shots_data],
            sequence_length=seq_len,
        )
        # Load real disruption labels from FAIR-MAST
        disruption_times = load_disruption_labels(loaded_ids)
        y = make_disruption_labels(loaded_ids, disruption_times)

        logger.info(
            "disruption_labels_loaded",
            n_disrupted=int(y.sum()),
            n_stable=int(len(y) - y.sum()),
        )

        from service.data.dataset import build_dataset
        dataset = build_dataset(X, y)
    else:
        # Use synthetic sample data for testing / development
        dataset = get_sample_dataset(
            n_samples=200,
            seq_len=seq_len,
            n_features=n_features,
            task=request.task,
        )

    # Train the model
    if request.model_type == "random_forest":
        X_tr, y_tr = dataset.flat_train()
        X_v, y_v = dataset.flat_val()
        X_te, y_te = dataset.flat_test()
    else:
        X_tr, y_tr = dataset.X_train, dataset.y_train
        X_v, y_v = dataset.X_val, dataset.y_val
        X_te, y_te = dataset.X_test, dataset.y_test

    train_metrics = model.fit(X_tr, y_tr, X_v, y_v)

    # Compute proper test metrics
    y_pred = model.predict(X_te)
    y_proba = model.predict_proba(X_te)
    test_metrics = compute_metrics(y_te, y_pred, y_proba)
    cm = format_confusion_matrix(y_te, y_pred)

    # Save model to persistent directory
    save_dir = "/tmp/tokamak_models"
    os.makedirs(save_dir, exist_ok=True)
    ext = ".joblib" if request.model_type == "random_forest" else ".pt"
    run_tag = f"_run{request.run_id}" if request.run_id else ""
    save_path = os.path.join(
        save_dir, f"{request.model_type}_{request.task}{run_tag}{ext}"
    )
    model.save(save_path)

```

```

        logger.info(
            "training_completed",
            model_type=request.model_type,
            test_metrics=test_metrics,
        )

    return {
        "status": "completed",
        "train_metrics": train_metrics,
        "test_metrics": test_metrics,
        "confusion_matrix": cm,
        "model_path": save_path,
        "metadata": model.metadata(),
    }
except HTTPException:
    raise
except Exception as e:
    logger.error("training_failed", error=str(e))
    raise HTTPException(status_code=500, detail=str(e))

```

```

def _get_model(model_type: str, model_path: str):
    """Load model with caching."""
    cache_key = f"{model_type}:{model_path}"
    if cache_key not in _model_cache:
        _model_cache[cache_key] = load_model(model_type, model_path)
    return _model_cache[cache_key]

```

=====

ФАЙЛ: ml_service/service/models/interface.py

Описание: Абстрактный интерфейс ModelInterface (протокол для всех ML-моделей)

=====

```

from __future__ import annotations

```

```

from abc import ABC, abstractmethod
from typing import Any

```

```

import numpy as np

```

```

class ModelInterface(ABC):

```

```

    """Unified interface for all ML models (ADR-004).

```

```

    All three models (Random Forest, bi-LSTM+attention, Transformer)
    implement this interface for interchangeability in the training
    and inference pipeline.
    """

```

```

    @abstractmethod

```

```

    def fit(
        self,
        X_train: np.ndarray,
        y_train: np.ndarray,
        X_val: np.ndarray | None = None,
        y_val: np.ndarray | None = None,
    )

```

```

        hyperparams: dict | None = None,
        progress_callback: Any = None,
    ) -> dict:
        """Train the model.

        Returns dict with training metrics (loss, accuracy, auc, etc.).
        """
        ...

    @abstractmethod
    def predict(self, X: np.ndarray) -> np.ndarray:
        """Return class labels (classification) or disruption probabilities over
        time."""
        ...

    @abstractmethod
    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        """Return probability estimates for AUC-ROC and calibration curves."""
        ...

    @abstractmethod
    def save(self, path: str) -> None:
        """Serialize model to file (joblib for RF, TorchScript for PyTorch
        models)."""
        ...

    @abstractmethod
    def load(self, path: str) -> "ModelInterface":
        """Load model from file."""
        ...

    @abstractmethod
    def metadata(self) -> dict:
        """Return model metadata: architecture, hyperparameters, training date,
        validation metrics, dataset hash."""
        ...

```

```

=====
ФАЙЛ: ml_service/service/models/random_forest.py
Описание: Модель Random Forest (базовая модель, scikit-learn)
=====

```

```

from __future__ import annotations

import hashlib
import json
from datetime import datetime, timezone
from typing import Any

import joblib
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

from service.models.interface import ModelInterface

```

```

class RandomForestModel(ModelInterface):
    """Random Forest baseline model (ADR-004 §3.2.1).

    Used as baseline for classification (FR-004).
    CPU inference <= 5ms. No GPU required.
    """

    RF_PARAMS = {
        "n_estimators", "max_depth", "min_samples_split", "min_samples_leaf",
        "class_weight", "max_features", "random_state", "n_jobs",
        "criterion", "min_weight_fraction_leaf", "max_leaf_nodes",
        "min_impurity_decrease", "bootstrap", "oob_score", "verbose",
        "warm_start", "ccp_alpha", "max_samples",
    }

    def __init__(self, hyperparams: dict | None = None):
        defaults = {
            "n_estimators": 200,
            "max_depth": 20,
            "min_samples_split": 5,
            "min_samples_leaf": 2,
            "class_weight": "balanced",
            "max_features": "sqrt",
            "random_state": 42,
            "n_jobs": -1,
        }
        if hyperparams:
            # Only keep params that RF actually accepts
            filtered = {k: v for k, v in hyperparams.items() if k in
self.RF_PARAMS}
            defaults.update(filtered)
        self.hyperparams = defaults
        self.model = RandomForestClassifier(**self.hyperparams)
        self.metrics: dict = {}
        self.trained_at: str | None = None
        self.dataset_hash: str | None = None

    def fit(
        self,
        X_train: np.ndarray,
        y_train: np.ndarray,
        X_val: np.ndarray | None = None,
        y_val: np.ndarray | None = None,
        hyperparams: dict | None = None,
        progress_callback: Any = None,
    ) -> dict:
        if hyperparams:
            self.hyperparams.update(hyperparams)
            self.model = RandomForestClassifier(**self.hyperparams)

        self.dataset_hash = hashlib.sha256(
            X_train.tobytes() + y_train.tobytes()
        ).hexdigest()[:16]

        self.model.fit(X_train, y_train)
        self.trained_at = datetime.now(timezone.utc).isoformat()

        self.metrics = {"train_accuracy": accuracy_score(y_train,

```

```

self.model.predict(X_train))}

    if X_val is not None and y_val is not None:
        y_pred = self.model.predict(X_val)
        y_proba = self.model.predict_proba(X_val)

        self.metrics.update({
            "val_accuracy": accuracy_score(y_val, y_pred),
            "val_f1": f1_score(y_val, y_pred, average="weighted"),
        })
        if y_proba.shape[1] == 2:
            self.metrics["val_auc_roc"] = roc_auc_score(y_val, y_proba[:, 1])

    return self.metrics

def predict(self, X: np.ndarray) -> np.ndarray:
    return self.model.predict(X)

def predict_proba(self, X: np.ndarray) -> np.ndarray:
    return self.model.predict_proba(X)

def save(self, path: str) -> None:
    artifact = {
        "model": self.model,
        "hyperparams": self.hyperparams,
        "metrics": self.metrics,
        "trained_at": self.trained_at,
        "dataset_hash": self.dataset_hash,
    }
    joblib.dump(artifact, path)

def load(self, path: str) -> "RandomForestModel":
    artifact = joblib.load(path)
    self.model = artifact["model"]
    self.hyperparams = artifact["hyperparams"]
    self.metrics = artifact["metrics"]
    self.trained_at = artifact["trained_at"]
    self.dataset_hash = artifact["dataset_hash"]
    return self

def metadata(self) -> dict:
    return {
        "architecture": "RandomForest",
        "hyperparameters": self.hyperparams,
        "trained_at": self.trained_at,
        "metrics": self.metrics,
        "dataset_hash": self.dataset_hash,
        "feature_importances": (
            self.model.feature_importances_.tolist()
            if hasattr(self.model, "feature_importances_")
            else None
        ),
    }
}

```

=====

ФАЙЛ: ml_service/service/models/lstm_attention.py

Описание: Модель bi-LSTM с multi-head attention (PyTorch)


```

=====

from __future__ import annotations

import hashlib
from datetime import datetime, timezone
from typing import Any

import numpy as np
import torch
import torch.nn as nn

from service.models.interface import ModelInterface

class AttentionLayer(nn.Module):
    """Multi-head attention over LSTM outputs."""

    def __init__(self, hidden_size: int, num_heads: int = 4):
        super().__init__()
        self.attention = nn.MultiheadAttention(
            embed_dim=hidden_size, num_heads=num_heads, batch_first=True
        )
        self.norm = nn.LayerNorm(hidden_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        attn_out, _ = self.attention(x, x, x)
        return self.norm(x + attn_out)

class BiLSTMAttentionNet(nn.Module):
    """Bidirectional LSTM with multi-head attention (ADR-004 §3.2.2).

    Based on HDL (Zhu et al., NC 2020) and FRNN (Kates-Harbeck et al., Nature
    2019).
    """

    def __init__(
        self,
        input_size: int = 39,
        hidden_size: int = 128,
        num_layers: int = 2,
        num_heads: int = 4,
        dropout: float = 0.3,
        bidirectional: bool = True,
    ):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=bidirectional,
            dropout=dropout if num_layers > 1 else 0.0,
        )
        lstm_output_size = hidden_size * (2 if bidirectional else 1)
        self.attention = AttentionLayer(lstm_output_size, num_heads)
        self.classifier = nn.Sequential(

```

```

        nn.Linear(lstm_output_size, 64),
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(64, 1),
    )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        lstm_out, _ = self.lstm(x)
        attn_out = self.attention(lstm_out)
        # Global average pooling over time
        pooled = attn_out.mean(dim=1)
        return self.classifier(pooled).squeeze(-1)

```

```

class LSTMAAttentionModel(ModelInterface):
    """bi-LSTM+attention model for classification and disruption prediction.

```

```

    Primary production model (ADR-004). GPU inference <= 30ms on T4.
    """

```

```

    def __init__(self, hyperparams: dict | None = None):
        self.hyperparams = hyperparams or {
            "input_size": 39,
            "hidden_size": 128,
            "num_layers": 2,
            "bidirectional": True,
            "dropout": 0.3,
            "attention_heads": 4,
            "sequence_length": 200,
            "learning_rate": 1e-3,
            "batch_size": 32,
            "epochs": 50,
            "early_stopping_patience": 10,
            "weight_decay": 1e-4,
        }
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = BiLSTMAAttentionNet(
            input_size=self.hyperparams["input_size"],
            hidden_size=self.hyperparams["hidden_size"],
            num_layers=self.hyperparams["num_layers"],
            num_heads=self.hyperparams["attention_heads"],
            dropout=self.hyperparams["dropout"],
            bidirectional=self.hyperparams["bidirectional"],
        ).to(self.device)
        self.metrics: dict = {}
        self.trained_at: str | None = None
        self.dataset_hash: str | None = None

    def fit(
        self,
        X_train: np.ndarray,
        y_train: np.ndarray,
        X_val: np.ndarray | None = None,
        y_val: np.ndarray | None = None,
        hyperparams: dict | None = None,
        progress_callback: Any = None,
    ) -> dict:
        if hyperparams:

```

```

        self.hyperparams.update(hyperparams)

self.dataset_hash = hashlib.sha256(X_train.tobytes()[:10000]).hexdigest()
[:16]

X_t = torch.FloatTensor(X_train).to(self.device)
y_t = torch.FloatTensor(y_train).to(self.device)

dataset = torch.utils.data.TensorDataset(X_t, y_t)
loader = torch.utils.data.DataLoader(
    dataset, batch_size=self.hyperparams["batch_size"], shuffle=True
)

optimizer = torch.optim.AdamW(
    self.model.parameters(),
    lr=self.hyperparams["learning_rate"],
    weight_decay=self.hyperparams["weight_decay"],
)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
factor=0.5, patience=5)

# Class weights for imbalanced data
pos_weight = torch.tensor([(y_train == 0).sum() / max((y_train == 1).sum(),
1)]).to(self.device)
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

best_val_loss = float("inf")
patience_counter = 0
history = {"train_loss": [], "val_loss": []}

self.model.train()
for epoch in range(self.hyperparams["epochs"]):
    epoch_loss = 0.0
    for X_batch, y_batch in loader:
        optimizer.zero_grad()
        output = self.model(X_batch)
        loss = criterion(output, y_batch)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.model.parameters(),
max_norm=1.0)
        optimizer.step()
        epoch_loss += loss.item()

    avg_loss = epoch_loss / len(loader)
    history["train_loss"].append(avg_loss)

    # Validation
    val_loss = avg_loss
    if X_val is not None and y_val is not None:
        self.model.eval()
        with torch.no_grad():
            X_v = torch.FloatTensor(X_val).to(self.device)
            y_v = torch.FloatTensor(y_val).to(self.device)
            val_output = self.model(X_v)
            val_loss = criterion(val_output, y_v).item()
        history["val_loss"].append(val_loss)
    self.model.train()

```

```

        scheduler.step(val_loss)

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            patience_counter = 0
        else:
            patience_counter += 1
            if patience_counter >= self.hyperparams["early_stopping_patience"]:
                break

        if progress_callback:
            progress_callback(epoch + 1, self.hyperparams["epochs"], avg_loss,
val_loss)

        self.trained_at = datetime.now(timezone.utc).isoformat()
        self.metrics = {
            "final_train_loss": history["train_loss"][-1],
            "best_val_loss": best_val_loss,
            "epochs_trained": len(history["train_loss"]),
            "history": history,
        }
        return self.metrics

def predict(self, X: np.ndarray) -> np.ndarray:
    self.model.eval()
    with torch.no_grad():
        X_t = torch.FloatTensor(X).to(self.device)
        output = torch.sigmoid(self.model(X_t))
        return (output.cpu().numpy() > 0.5).astype(int)

def predict_proba(self, X: np.ndarray) -> np.ndarray:
    self.model.eval()
    with torch.no_grad():
        X_t = torch.FloatTensor(X).to(self.device)
        proba = torch.sigmoid(self.model(X_t)).cpu().numpy()
        return np.column_stack([1 - proba, proba])

def save(self, path: str) -> None:
    torch.save({
        "model_state_dict": self.model.state_dict(),
        "hyperparams": self.hyperparams,
        "metrics": self.metrics,
        "trained_at": self.trained_at,
        "dataset_hash": self.dataset_hash,
    }, path)

def load(self, path: str) -> "LSTMAttentionModel":
    checkpoint = torch.load(path, map_location=self.device, weights_only=False)
    self.hyperparams = checkpoint["hyperparams"]
    self.model = BiLSTMAttentionNet(
        input_size=self.hyperparams["input_size"],
        hidden_size=self.hyperparams["hidden_size"],
        num_layers=self.hyperparams["num_layers"],
        num_heads=self.hyperparams["attention_heads"],
        dropout=self.hyperparams["dropout"],
        bidirectional=self.hyperparams["bidirectional"],
    ).to(self.device)
    self.model.load_state_dict(checkpoint["model_state_dict"])

```

```

        self.metrics = checkpoint["metrics"]
        self.trained_at = checkpoint["trained_at"]
        self.dataset_hash = checkpoint["dataset_hash"]
        return self

    def metadata(self) -> dict:
        return {
            "architecture": "BiLSTM_Attention",
            "hyperparameters": self.hyperparams,
            "trained_at": self.trained_at,
            "metrics": self.metrics,
            "dataset_hash": self.dataset_hash,
            "device": str(self.device),
            "total_parameters": sum(p.numel() for p in self.model.parameters()),
        }

```

```

=====
ФАЙЛ: ml_service/service/models/transformer.py
Описание: Autoregressive Transformer с RoPE (PyTorch)
=====

```

```

from __future__ import annotations

```

```

import hashlib
import math
from datetime import datetime, timezone
from typing import Any

```

```

import numpy as np
import torch
import torch.nn as nn

```

```

from service.models.interface import ModelInterface

```

```

class RotaryPositionalEncoding(nn.Module):
    """Rotary Position Embedding (RoPE)."""

```

```

    def __init__(self, d_model: int, max_len: int = 5000):
        super().__init__()
        position = torch.arange(max_len).unsqueeze(1).float()
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-
math.log(10000.0) / d_model))
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe.unsqueeze(0))

```

```

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x + self.pe[:, : x.size(1)]

```

```

class AutoregressiveTransformer(nn.Module):
    """Autoregressive Transformer for disruption prediction (ADR-004 §3.2.3).

```

```

    Based on Spangher et al. (arXiv:2401.00051).
    Causal attention mask for autoregressive prediction.

```

```

"""

def __init__(
    self,
    input_size: int = 39,
    d_model: int = 128,
    num_heads: int = 8,
    num_layers: int = 4,
    dim_feedforward: int = 512,
    dropout: float = 0.1,
    max_seq_len: int = 1000,
):
    super().__init__()
    self.input_proj = nn.Linear(input_size, d_model)
    self.pos_encoding = RotaryPositionalEncoding(d_model, max_seq_len)
    self.dropout = nn.Dropout(dropout)

    encoder_layer = nn.TransformerEncoderLayer(
        d_model=d_model,
        nhead=num_heads,
        dim_feedforward=dim_feedforward,
        dropout=dropout,
        batch_first=True,
        activation="gelu",
    )
    self.transformer = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)
    self.output_head = nn.Sequential(
        nn.Linear(d_model, d_model // 2),
        nn.GELU(),
        nn.Dropout(dropout),
        nn.Linear(d_model // 2, 1),
    )

    def _generate_causal_mask(self, sz: int, device: torch.device) -> torch.Tensor:
        return torch.triu(torch.ones(sz, sz, device=device) * float("-inf"),
diagonal=1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.input_proj(x)
        x = self.pos_encoding(x)
        x = self.dropout(x)

        mask = self._generate_causal_mask(x.size(1), x.device)
        x = self.transformer(x, mask=mask)

        # Output per-timestep predictions
        return self.output_head(x).squeeze(-1)

class TransformerModel(ModelInterface):
    """Autoregressive Transformer for disruption prediction.

    Experimental model (ADR-004). GPU inference <= 50ms on T4.
    """

    def __init__(self, hyperparams: dict | None = None):
        self.hyperparams = hyperparams or {

```

```

        "input_size": 39,
        "d_model": 128,
        "num_heads": 8,
        "num_layers": 4,
        "dim_feedforward": 512,
        "dropout": 0.1,
        "sequence_length": 500,
        "learning_rate": 5e-4,
        "warmup_steps": 1000,
        "batch_size": 16,
        "epochs": 40,
        "early_stopping_patience": 8,
        "weight_decay": 1e-4,
    }
    self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    self.model = AutoregressiveTransformer(
        input_size=self.hyperparams["input_size"],
        d_model=self.hyperparams["d_model"],
        num_heads=self.hyperparams["num_heads"],
        num_layers=self.hyperparams["num_layers"],
        dim_feedforward=self.hyperparams["dim_feedforward"],
        dropout=self.hyperparams["dropout"],
        max_seq_len=self.hyperparams["sequence_length"],
    ).to(self.device)
    self.metrics: dict = {}
    self.trained_at: str | None = None
    self.dataset_hash: str | None = None

```

```

def fit(
    self,
    X_train: np.ndarray,
    y_train: np.ndarray,
    X_val: np.ndarray | None = None,
    y_val: np.ndarray | None = None,
    hyperparams: dict | None = None,
    progress_callback: Any = None,
) -> dict:
    if hyperparams:
        self.hyperparams.update(hyperparams)

```

```

    self.dataset_hash = hashlib.sha256(X_train.tobytes()[:10000]).hexdigest()

```

[:16]

```

X_t = torch.FloatTensor(X_train).to(self.device)
y_t = torch.FloatTensor(y_train).to(self.device)

dataset = torch.utils.data.TensorDataset(X_t, y_t)
loader = torch.utils.data.DataLoader(
    dataset, batch_size=self.hyperparams["batch_size"], shuffle=True
)

optimizer = torch.optim.AdamW(
    self.model.parameters(),
    lr=self.hyperparams["learning_rate"],
    weight_decay=self.hyperparams["weight_decay"],
)

# Cosine annealing with warmup

```

```

def lr_lambda(step):
    if step < self.hyperparams["warmup_steps"]:
        return step / max(self.hyperparams["warmup_steps"], 1)
    progress = (step - self.hyperparams["warmup_steps"]) / max(
        len(loader) * self.hyperparams["epochs"] -
self.hyperparams["warmup_steps"], 1
    )
    return 0.5 * (1 + math.cos(math.pi * progress))

scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)
criterion = nn.BCEWithLogitsLoss()

best_val_loss = float("inf")
patience_counter = 0
history = {"train_loss": [], "val_loss": []}

self.model.train()
for epoch in range(self.hyperparams["epochs"]):
    epoch_loss = 0.0
    for X_batch, y_batch in loader:
        optimizer.zero_grad()
        output = self.model(X_batch)
        # Use last timestep prediction for classification
        if output.dim() > 1:
            output = output[:, -1]
        loss = criterion(output, y_batch)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.model.parameters(),
max_norm=1.0)
        optimizer.step()
        scheduler.step()
        epoch_loss += loss.item()

    avg_loss = epoch_loss / len(loader)
    history["train_loss"].append(avg_loss)

    val_loss = avg_loss
    if X_val is not None and y_val is not None:
        self.model.eval()
        with torch.no_grad():
            X_v = torch.FloatTensor(X_val).to(self.device)
            y_v = torch.FloatTensor(y_val).to(self.device)
            val_output = self.model(X_v)
            if val_output.dim() > 1:
                val_output = val_output[:, -1]
            val_loss = criterion(val_output, y_v).item()
        history["val_loss"].append(val_loss)
        self.model.train()

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= self.hyperparams["early_stopping_patience"]:
            break

    if progress_callback:

```



```

        progress_callback(epoch + 1, self.hyperparams["epochs"], avg_loss,
val_loss)

        self.trained_at = datetime.now(timezone.utc).isoformat()
        self.metrics = {
            "final_train_loss": history["train_loss"][-1],
            "best_val_loss": best_val_loss,
            "epochs_trained": len(history["train_loss"]),
        }
        return self.metrics

def predict(self, X: np.ndarray) -> np.ndarray:
    self.model.eval()
    with torch.no_grad():
        X_t = torch.FloatTensor(X).to(self.device)
        output = torch.sigmoid(self.model(X_t))
        if output.dim() > 1:
            output = output[:, -1]
        return (output.cpu().numpy() > 0.5).astype(int)

def predict_proba(self, X: np.ndarray) -> np.ndarray:
    self.model.eval()
    with torch.no_grad():
        X_t = torch.FloatTensor(X).to(self.device)
        output = torch.sigmoid(self.model(X_t))
        if output.dim() > 1:
            output = output[:, -1]
        proba = output.cpu().numpy()
        return np.column_stack([1 - proba, proba])

def save(self, path: str) -> None:
    torch.save({
        "model_state_dict": self.model.state_dict(),
        "hyperparams": self.hyperparams,
        "metrics": self.metrics,
        "trained_at": self.trained_at,
        "dataset_hash": self.dataset_hash,
    }, path)

def load(self, path: str) -> "TransformerModel":
    checkpoint = torch.load(path, map_location=self.device, weights_only=False)
    self.hyperparams = checkpoint["hyperparams"]
    self.model = AutoregressiveTransformer(
        input_size=self.hyperparams["input_size"],
        d_model=self.hyperparams["d_model"],
        num_heads=self.hyperparams["num_heads"],
        num_layers=self.hyperparams["num_layers"],
        dim_feedforward=self.hyperparams["dim_feedforward"],
        dropout=self.hyperparams["dropout"],
        max_seq_len=self.hyperparams["sequence_length"],
    ).to(self.device)
    self.model.load_state_dict(checkpoint["model_state_dict"])
    self.metrics = checkpoint["metrics"]
    self.trained_at = checkpoint["trained_at"]
    self.dataset_hash = checkpoint["dataset_hash"]
    return self

def metadata(self) -> dict:

```

```

return {
  "architecture": "AutoregressiveTransformer",
  "hyperparameters": self.hyperparams,
  "trained_at": self.trained_at,
  "metrics": self.metrics,
  "dataset_hash": self.dataset_hash,
  "device": str(self.device),
  "total_parameters": sum(p.numel() for p in self.model.parameters()),
}

```

=====

ФАЙЛ: frontend/src/App.tsx

Описание: Корневой компонент React-приложения, маршрутизация

=====

```

import { BrowserRouter, Navigate, Route, Routes } from 'react-router-dom';
import { useAuthStore } from './store/authStore';
import Layout from './components/Layout';
import ToastContainer from './components/Toast';
import LoginPage from './pages/LoginPage';
import DashboardPage from './pages/DashboardPage';
import ExperimentPage from './pages/ExperimentPage';
import TrainingPage from './pages/TrainingPage';
import BatchPage from './pages/BatchPage';
import SettingsPage from './pages/SettingsPage';

```

```

function ProtectedRoute({ children }: { children: React.ReactNode }) {
  const isAuthenticated = useAuthStore((s) => s.isAuthenticated);
  return isAuthenticated ? <>{children}</> : <Navigate to="/login" />;
}

```

```

export default function App() {
  return (
    <BrowserRouter>
      <ToastContainer />
      <Routes>
        <Route path="/login" element={<LoginPage />} />
        <Route
          path="/"
          element={
            <ProtectedRoute>
              <Layout />
            </ProtectedRoute>
          }
        />
        <Route index element={<DashboardPage />} />
        <Route path="experiment/:id" element={<ExperimentPage />} />
        <Route path="training" element={<TrainingPage />} />
        <Route path="batch" element={<BatchPage />} />
        <Route path="settings" element={<SettingsPage />} />
      </Routes>
    </BrowserRouter>
  );
}

```

=====

ФАЙЛ: frontend/src/pages/DashboardPage.tsx

Описание: Главная страница – статистика, загрузка данных, сравнение моделей

=====

```
import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import Plot from 'react-plotly.js';
import api from '../api/client';
import ShotLoader from '../components/ShotLoader';
import type { Experiment, MLModel } from '../types';

export default function DashboardPage() {
  const [experiments, setExperiments] = useState<Experiment[]>([]);
  const [models, setModels] = useState<MLModel[]>([]);
  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();

  useEffect(() => {
    api
      .get('/experiments/')
      .then((res) => setExperiments(res.data.experiments))
      .catch(() => {})
      .finally(() => setLoading(false));

    api
      .get('/models/')
      .then((res) => setModels(res.data))
      .catch(() => {});
  }, []);

  const handleLoaded = (exp: Experiment) => {
    setExperiments((prev) => [exp, ...prev]);
    navigate(`/experiment/${exp.id}`);
  };

  const preprocessedCount = experiments.filter((e) => e.status ===
'preprocessed').length;
  const totalSignals = experiments.reduce(
    (sum, e) => sum + ((e.metadata_json?.signal_count as number) || 0),
    0
  );

  const statusBadge = (status: string) => {
    const cls =
      status === 'preprocessed'
        ? 'badge-success'
        : status === 'error'
        ? 'badge-danger'
        : 'badge-warning';
    return <span className={`badge ${cls}`}>{status}</span>;
  };

  return (
    <div>
      <h2 className="mb-lg">Dashboard</h2>

      {/* Summary Stats */}
```

```

<div className="grid-3 mb-lg">
  <div className="stat-card">
    <div className="stat-icon stat-icon-primary">{'\u26A1'}</div>
    <div>
      <div className="stat-value">{experiments.length}</div>
      <div className="stat-label">Экспериментов</div>
    </div>
  </div>
  <div className="stat-card">
    <div className="stat-icon stat-icon-success">{'\u2713'}</div>
    <div>
      <div className="stat-value">{preprocessedCount}</div>
      <div className="stat-label">Обработано</div>
    </div>
  </div>
  <div className="stat-card">
    <div className="stat-icon stat-icon-warning">{'\u223F'}</div>
    <div>
      <div className="stat-value">{totalSignals}</div>
      <div className="stat-label">Сигналов</div>
    </div>
  </div>
</div>

```

```

<ShotLoader onLoaded={handleLoaded} />

```

```

{/* Model Comparison */}

```

```

{models.length > 0 && (
  <div className="card mb-lg">
    <div className="card-header">
      <h3>Сравнение моделей</h3>
    </div>

```

```

    <table className="table" style={{ marginBottom: 20 }}>

```

```

      <thead>

```

```

        <tr>

```

```

          <th>Модель</th>

```

```

          <th>Тип</th>

```

```

          <th>Accuracy</th>

```

```

          <th>F1</th>

```

```

          <th>AUC</th>

```

```

          <th>Статус</th>

```

```

        </tr>

```

```

      </thead>

```

```

      <tbody>

```

```

        {models.map((m) => (

```

```

          <tr key={m.id}>

```

```

            <td className="font-semibold">{m.name}</td>

```

```

            <td>

```

```

              <span className="badge badge-info">{m.model_type}</span>

```

```

            </td>

```

```

            <td>{m.metrics_json?.accuracy != null ?

```

```

m.metrics_json.accuracy.toFixed(4) : '\u2014'}</td>

```

```

            <td>{m.metrics_json?.f1 != null ? m.metrics_json.f1.toFixed(4) :

```

```

'\u2014'}</td>

```

```

            <td>{m.metrics_json?.auc != null ?

```

```

m.metrics_json.auc.toFixed(4) : '\u2014'}</td>

```

```

            <td>

```

```

        <span className={`badge ${m.is_active ? 'badge-success' :
'badge-neutral'}}`>
            {m.is_active ? 'Активна' : 'Неактивна'}
        </span>
    </td>
</tr>
</tbody>
</table>

{/* AUC bar chart */}
{models.some((m) => m.metrics_json?.auc != null) && (
    <Plot
        data={[
            {
                x: models.filter((m) => m.metrics_json?.auc != null).map((m) =>
m.name),
                y: models.filter((m) => m.metrics_json?.auc != null).map((m) =>
m.metrics_json!.auc),
                type: 'bar',
                marker: {
                    color: models
                        .filter((m) => m.metrics_json?.auc != null)
                        .map((m) => (m.is_active ? '#2ecc71' : '#3498db')),
                },
                hovertemplate: '%{x}<br>AUC: %{y:.4f}<extra></extra>',
            },
        ]}
        layout={{
            title: 'Сравнение AUC моделей',
            yaxis: { title: 'AUC', range: [0, 1] },
            height: 350,
            margin: { l: 60, r: 30, t: 50, b: 80 },
        }}
        config={{ responsive: true, displayModeBar: false }}
        style={{ width: '100%' }}
    />
)}
</div>
)}

<div className="card">
    <div className="card-header">
        <h3>Загруженные эксперименты</h3>
    </div>

    {loading ? (
        <div className="loading-overlay">
            <span className="spinner spinner-lg" />
            Загрузка данных...
        </div>
    ) : experiments.length === 0 ? (
        <div className="empty-state">
            <div className="empty-state-icon">{'\u269B'}</div>
            <div className="empty-state-text">Нет загруженных экспериментов</div>
            <div className="empty-state-hint">
                Загрузите данные выстрела, указав Shot ID выше
            </div>
        </div>
    ) : experiments.length > 0 ? (
        <div className="experiments">
            {experiments.map((exp) => (
                <div className="experiment-card">
                    <div>{exp.name}</div>
                    <div>{exp.metrics_json?.auc}</div>
                </div>
            ))}
        </div>
    ) : null}
    </div>

```

```

    </div>
  ) : (
    <table className="table">
      <thead>
        <tr>
          <th>Shot ID</th>
          <th>Источник</th>
          <th>Статус</th>
          <th>Дата</th>
          <th>Сигналов</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {experiments.map((exp) => (
          <tr key={exp.id}>
            <td className="font-semibold">{exp.shot_id}</td>
            <td>
              <span className="badge badge-info">{exp.source.toUpperCase()}</span>
            </td>
            <td>{statusBadge(exp.status)}</td>
            <td className="text-sm text-secondary">
              {new Date(exp.loaded_at).toLocaleString('ru')}
            </td>
            <td>{(exp.metadata_json?.signal_count as number) ?? '\u2014'}</td>
            <td>
              <button
                className="btn btn-primary btn-sm"
                onClick={() => navigate(`/experiment/${exp.id}`)}
              >
                Открыть
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  )}
</div>
</div>
);
}

```

```

=====
КОНЕЦ ДЕПОНИРУЕМЫХ МАТЕРИАЛОВ
Всего файлов: 10
=====

```